ARCHEA Platform

Datasheet del codice

Come è fatto, come funziona, dove sono le funzioni. Generato il 07/11/2026 da `tools/genera_datasheet.py`.

Francesco Diamanti · Giuseppe Milano

1. Mappa del progetto

Parte	Tecnologia	Cosa fa
`index.html` + `assets/workspace.js`	React 18 compilato (Francesco)	Guscio della piattaforma: menu, Oggi, Task Manager, Progetti, Organico…
`servicedesk/`	JavaScript senza framework	Ticket, asset, HR, planimetrie, calendario, chat, Off topic, foto, fornitori
`assets/sd-shell.js` , `ws-cloud.js`	JavaScript	Collegano Workspace, Service Desk e database
`app/` → `v2/`	React + TypeScript + Vite	Nuova versione: qui vanno le funzioni nuove
`supabase/`	PostgreSQL, RLS, Edge Functions	Tabelle, regole di accesso, funzioni, notifiche
`sw.js`	Service worker	App sul telefono, offline, notifiche push

2. Come funziona: i flussi principali

Avvio e accesso

1. index.html carica config/platform.js (ArcheaAuth, ArcheaTheme), assets/workspace.js (React di Francesco) e assets/sd-shell.js.
2. ArcheaAuth.signIn() fa il login su Supabase Auth e salva la sessione in localStorage «archea_sd_session»; sessionStorage «archea-live» = 1 indica un account vero.
3. workspace.js: Bv() mostra l'animazione solo a chi non è ancora entrato; poi il guscio con il menu.
4. sd-shell.js crea UNA sola iframe del Service Desk (servicedesk/?embed=1&shell=1) e la appoggia sopra il posto «.sd-slot»: cambiando sezione non si ricarica.
5. servicedesk/app.js → boot(): se il token è valido e il profilo è in sessionStorage (archea_sd_boot) entra subito con startApp(); loadIdentity() ricontrolla in silenzio.
6. applyRoleNav() mostra la barra del ruolo e chiama window.parent.archeaSdReady(); firstView() sceglie la prima pagina; il PM viene portato su «Oggi».

Navigazione

1. Barra in alto (Service Desk): pulsanti con data-view → nav(v) in app.js disegna la vista.
2. Menu a sinistra: sd-shell.js navData() raggruppa le sezioni della piattaforma (window.__archeaTabs); run(key) cambia sezione (location.hash) o chiede al Service Desk nav(…).
3. Chat, Calendario, Off topic, Foto: la classe «sd-free» nasconde la barra in alto.

Ticket

1. newTicket() → insert('tickets') → update numero_ticket → api('/functions/v1/telegram-new-ticket') → insert('ticket_history').
2. Coda IT: it(); scheda: detail(id) con «Prendi in carico», stato, nota, collaboratori, commenti (comments).
3. I trigger del database creano le notifiche (notifications) per le persone giuste.

Notifiche

1. Nel sito: refreshNotifications() ogni 15 s → notifPopups() → riquadro + notifica di sistema.
2. A sito chiuso: trigger push_on_notification (SQL) → pg_net → Edge Function «push» → Web Push cifrato → sw.js evento «push» → showNotification().
3. Iscrizione del dispositivo: push.js archeaEnablePush() → Notification.requestPermission() → pushManager.subscribe() → rpc push_claim. Le chiavi VAPID le crea il Super admin al primo accesso (push_setup).

Progetti, task, ore (Workspace)

1. Le schermate di Francesco usano useWs/useWsMap, collegati a ArcheaWsCloud (ws-cloud.js).
2. ArcheaWsCloud.start() → loadPeople() (permessi, colleghi, PM) → loadAll() legge ws_records ogni 20 s.
3. set(collezione, righe) → persist(): POST nuove, PATCH cambiate, DELETE tolte. Errore di permesso = messaggio in basso.
4. Nel database: RLS su ws_records (il PM vede tutto, l'utente le sue task), trigger ws_guard (i campi del PM non si cambiano), chat_sync_project (chat del progetto = squadra).

Spostamento sulla planimetria

1. plSeatPanel() → «Assegna» → plConfirmMove(): conferma 1 e conferma 2.
2. plRecordMove(): plSetSeat() (people.map_position_id + person_events) → insert('hr_movements') SPOSTAMENTO effettuato → rilettura di people e hr_movements: se non torna, errore.

Nuova versione React (app/)

1. main.tsx crea React Query e monta App; App.tsx: HashRouter + AuthProvider + rotte.
2. lib/auth.tsx: la sessione si legge una volta (getSession) e poi si ascolta (onAuthStateChange); profilo e permessi in memoria 5 minuti.
3. lib/roles.ts: ruoli, permessi, gruppiMenu(), primaPagina() (testati in roles.test.ts).
4. lib/tickets.ts: accesso ai ticket; pages/*: una pagina per file.

3. Le funzioni, file per file

In grassetto le funzioni principali con la loro descrizione. Il numero è la riga del file: aprilo e cerca il nome.

`servicedesk/app.js` 3604 righe · 123 funzioni

Cuore del Service Desk: accesso, chiamate al database, ticket, HR, asset, navigazione.

Funzione	Riga	Cosa fa
xlsEsc(v)	55	V8 — ordinamento tabelle + export Excel XML (.xls)
enableTableTools(root=document)	76	Aggiunge a ogni tabella ordinamento per colonna e pulsante «Scarica XLS».
toast(t)	193	Messaggio breve in basso.
page(t,s='')	194	Imposta titolo e sottotitolo della pagina.
save(s)	195	Salva la sessione in localStorage (archea_sd_session), condivisa con la piattaforma.
clear()	199	Cancella sessione e memoria di avvio (uscita).
refreshSession()	207	Rinnova il token di accesso con il refresh token; se un'altra scheda l'ha già rinnovato usa quello.
showEmbedSignedOut(msg)	251	Dentro il Workspace l'accesso è unico: niente form di login qui.
api(path, {method='GET',body=null,auth=true,h)	268	Unico punto che parla con Supabase: aggiunge chiave e token, rinnova la sessione se scade (401) e trasforma gli errori in messaggi.
select(t,q='')	306	Legge righe da una tabella (GET /rest/v1/tabella?filtri).
selectAll(t,q='',pageSize=1000)	307	Come select ma a pagine da 1000: per tabelle grandi (asset, persone).
insert(t,b,ret=true)	319	Inserisce righe (POST). Con ret=true restituisce le righe create.
update(t,f,b)	320	Modifica le righe che rispettano il filtro (PATCH).
rpc(fn,b={})	321	Chiama una funzione SQL del database (POST /rest/v1/rpc/nome).
notifPopups(list)	328	Mostra i riquadri delle notifiche nuove e, se permesso, la notifica di sistema.
enableSystemNotifications()	356	Pulsante della campanella: passa a archeaEnablePush (push.js).
refreshNotifications()	364	Ogni 15 secondi legge le notifiche dell'utente, aggiorna la campanella e i popup.
table(rows,it=false,quick=false)	377	Disegna una tabella di ticket (con ordinamento ed esportazione Excel).
isITRole()	419	Vero per IT e SUPER_IT.
isSuperIT()	420	Vero solo per il Super admin.
isHR()	421	Vero per HR.
normSearch(s)	458	Normalizza un testo per la ricerca (minuscole, senza accenti).
userHome()	496	Prima pagina dell'utente: le sue richieste e «Apri un ticket».
legacyUserHome()	497	Disegna la pagina «Service Desk · Come possiamo aiutarti?».
home()	498	Dashboard IT: contatori, carico del team, ticket più vecchi.
newTicket()	545	Modulo «Nuovo ticket»: crea il ticket, il numero TKT-AAAA-NNNNN, avvisa Telegram e scrive lo storico.
mine()	712	Elenco «I miei ticket».
it()	713	Coda «Gestione IT» con filtri e «Prendi».
calendar()	816	Appuntamenti IT.
bookings()	817	Prenotazioni di materiale.
hrNewMovement()	1131	Modulo HR «Nuovo movimento»: controlla doppioni, crea il ticket IT (MOV-…) e la riga in hr_movements.
loadUnifiedHrMovements()	1547	Unisce i movimenti del portale (hr_movements) e lo storico HR importato (person_events).
hrHistory()	1630	Pagina «Movimenti / Storico movimenti».
hrStats()	1731	Statistiche HR (prima pagina dell'HR).
people()	1826	Elenco persone (anagrafica).
personDetail(id)	2101	Scheda persona: dati, asset collegati, licenze e programmi (IT), storico.
personEdit(id)	2195	Correzione dei dati di una persona (non è un movimento).
census()	2226	Censimento asset con lettore di codici.
assetEdit(id,changeMode='CORREZIONE')	2499	Modifica asset.
assetDetail(id)	2787	Scheda asset con storico.
detail(id)	2910	Scheda di un ticket: gestione IT, commenti, appuntamento, checklist.
openAssetScanner(cachedRows=null,opts={})	3070	Apre il lettore di codici a barre/QR (fotocamera, foto, iPhone come lettore).
getBarcodeDetector()	3193	Lettore nativo o, se manca, quello ZXing incluso (vendor/).
makeFrameReader(video)	3203	Legge un fotogramma alternando: riquadro centrale, riquadro ingrandito 2x (codici piccoli o lontani), immagine intera. Contrasto aumentato per le webcam dei computer, che non mettono a fuoco da vicino.
readCodeFromFile(file)	3226	Legge il codice da una foto (file): utile da computer o quando la fotocamera dal vivo non mette a fuoco
openSerialScanner(targetInput)	3249	V8.2 — SCANSIONE SERIALE: BARCODE + TESTO (OCR) + MANUALE
mapView()	3296	V8 — MAPPA SCHEMATICA POSTAZIONI / MAGAZZINO
setActiveNavigation(view)	3391	Evidenzia il pulsante della vista attiva.
nav(v)	3399	Cambia vista del Service Desk: decide quale funzione disegna la pagina e controlla il ruolo.

loadIdentity()	3468	Legge utente, profilo e permessi e li salva per l'avvio veloce.
showSignedOut(msg)	3476	Mostra il login (solo se la sessione non è valida).
boot()	3481	Avvio: se la sessione è valida e il profilo è in memoria entra subito (niente login), poi ricontrolla in silenzio.
applyRoleNav()	3499	Mostra la barra delle funzioni del proprio ruolo e avvisa la piattaforma (archeaSdReady).
firstView()	3506	Prima pagina per ruolo: IT → Dashboard, HR → Statistiche HR, utente → le sue richieste.
startApp()	3512	Mostra l'app, apre la prima pagina e i collegamenti diretti (?ticket=, ?scan=).

Altre funzioni di supporto: applyTheme (16), syncThemeLabel (23), initTheme (24), updateBackButton (30), openSubView (35), goBack (44), safeFileName (56), downloadXls (57), tableRowsForXls (66), inferSortValue (69), warehouseCodeOk (101), normalPositionCodeOk (102), enhanceSelect (104), showSessionExpired (257), readStoredSession (267), fileToBase64 (322), notify (323), profileByEmail (324), allIT (325), syncNotifButton (363), wire (403), dateOnly (407), bookingStatusBadge (408), currentITName (418), isHROrIT (422), canImportHR (423), canManageHRValues (424), canApproveHR (425), canRequestConflictResolution (426), canConfirmConflictResolution (427), assetStatusBadge (428), verifyBadge (442), assetTypeBadge (453), assetSearchNeedles (461), matchesAssetSearch (475), relTime (481), ageClass (489), bookingDetail (893), printDeliverySheet (1074), window.onload (1125), hrMovementKind (1537), hrRawStatusActive (1545), personStatusBadge (1812), personVerifyBadge (1816), sourceBadge (1820), sameText (1824), personCreate (1982), referenceValues (2051), legacyPersonCreate (2082), assetPersonRow (2157), sourceRecordHtml (2158), personHistoryHtml (2164), deletionRequests (2874), placeholder (2900), comments (3043), checks (3055), openCode (3097), improveCamera (3240), showPositionEditor (3340), showAssignPerson (3344), navIconKey (3381), installNavIcons (3382), window.sdCurrentView (3455), jwtSub (3465), saveBootCache (3466), readBootCache (3467), closeMobileNav (3532), openMobileNav (3541)

`servicedesk/start-chat.js` 499 righe · 23 funzioni

Gruppi di funzioni e chat (dirette, gruppi, progetti, sottochat).

Funzione	Riga	Cosa fa
startGroups()	49	Gruppi di funzioni per ruolo (usati dal menu della piattaforma).
window.sdNavGroups()	66	per la barra laterale della piattaforma: gruppi e voci con le loro etichette
startAction(key)	155	Esegue una funzione dal suo codice (es. "new", "ws:taskmanager").
chatLoad()	194	Carica contatti, messaggi diretti, gruppi, sottochat e membri.
chatRender()	256	Disegna la chat: Progetti (con sottochat), Gruppi, Colleghi e la conversazione aperta.
chatGroupForm(t, parent)	354	Crea o modifica un gruppo o una sottochat di progetto (nome e persone).
chatSend(e)	441	Invia un messaggio (diretto o di gruppo).
chatPoll()	463	Ricarica i messaggi ogni pochi secondi.
chatView(openWith)	479	Entrata nella chat.

Altre funzioni di supporto: startActionsForRole (64), startActionsBase (70), greetingWord (72), start (77), chatStop (192), chatThreads (213), chatGroupThreads (230), chatName (241), chatInitials (246), chatTime (249), chatMarkRead (409), chatMarkGroupRead (417), chatOpen (426), chatOpenGroup (433)

`servicedesk/agenda.js` 532 righe · 32 funzioni

Calendario comune, sale e auto.

Funzione	Riga	Cosa fa
agLoad()	68	Carica eventi, risorse e partecipanti.
agRange()	103	Intervallo visibile per la vista scelta (giorno, settimana, mese)
agRenderTimeGrid(days)	141	Griglia oraria (giorno = 1 colonna, settimana = 7 colonne)
agRenderMonth(compact)	169	Vista mese: 6 settimane, al massimo 3 eventi per giorno + "altri N"
agendaView(opts = {})	291	Calendario comune.
agResourcesAdmin()	303	Gestione sale e auto (permesso "gestione_calendario")
agForm(tipo = 'APPUNTAMENTO', preset = {})	354	Modulo appuntamento / prenotazione sala o auto (un clic su una casella lo apre).
agDetail(id)	486	Dettaglio di un appuntamento con accetta/rifiuta.

Altre funzioni di supporto: agSafeUrl (26), agMonday (36), agAddDays (41), agYmd (42), agHm (43), agDayLabel (44), agSameDay (47), agCanBook (48), agGroupName (53), agResource (54), agAttendeesOf (55), agContactName (56), agFriendlyError (61), agVisible (86), agEventChip (91), agShift (114), agRangeLabel (121), agDaysOfRange (129), agEventsOn (134), agRenderList (198), agRender (209), agRefresh (278), agSetupMessage (284), agDefaultSlot (348)

`servicedesk/offtopic.js` 346 righe · 12 funzioni

Off topic: argomenti, risposte, sondaggi.

Funzione	Riga	Cosa fa
offtopicView()	36	Bacheca Off topic.
otForm()	85	Nuovo argomento o modifica.
otOpen(id)	159	Argomento con risposte e sondaggio.

Altre funzioni di supporto: otWhen (16), otCanModerate (23), otMine (24), otSetupMessage (25), otBody (31), otRenderList (48), otPollOpen (300), otPollHtml (302), otWirePoll (328)

`servicedesk/users.js` 311 righe · 16 funzioni

Permessi e gestione account.

Funzione	Riga	Cosa fa

loadPerms()	23	Legge i permessi dei gruppi (my_permissions).
hasPerm(p)	35	Vero se l'utente ha il permesso (il Super admin li ha tutti).
adminUsers(action, payload = {})	44	Chiama la funzione SQL admin_users (o la Edge Function di riserva).
usersView()	67	Utenti e gruppi.
usForm(u, prefill = null)	139	Crea o modifica un account (ruolo, gruppi, password temporanea).
window.usAddAccount()	306	Da altre schermate (es. Persone): apre "Aggiungi utente" già compilato

Altre funzioni di supporto: applyPermNav (36), usSetupMessage (58), usGroupName (84), usStatus (85), usRender (93), usWireTabs (131), usRandomPassword (133), usGroupsHtml (230), usWireGroups (244), usGroupForm (248)

`servicedesk/planimetrie.js` 672 righe · 46 funzioni

Planimetrie, postazioni, spostamenti.

Funzione	Riga	Cosa fa
planimetrieView()	151	Planimetrie: chi siede dove, dispositivi, editor delle piante.
plEnsurePositions(plans)	197	crea in map_positions le postazioni che mancano (stesso codice della pianta)
plSeatPanel(e)	308	Pannello di una postazione: assegna o libera.
plConfirmMove(m, back)	339	Spostamento persona: conferma 1, conferma 2, poi scrittura e verifica.
plRecordMove(person, to, note)	376	Scrive posto + movimento (hr_movements, SPOSTAMENTO effettuato) e rilegge dal database per verificarlo.
plSetSeat(person, pos)	408	Aggiorna people.map_position_id e lo storico della persona.
plMoveTo(target)	446	Sposta un dispositivo su un'altra postazione (asset_history).
plSavePlan()	621	Salva la pianta disegnata.

Altre funzioni di supporto: plCanEdit (43), plUnit (44), plSnap (45), plUid (46), plName (47), plSlug (48), plPos (51), plOccupant (58), plDevices (62), plElSvg (70), plSvg (127), plLoadData (141), plEmpty (171), plTemplates (179), plImportAll (184), plRender (214), plToolbarHtml (240), plHintText (255), plPropsHtml (261), plWire (278), plWireUse (299), plPosById (334), plMoveSummary (335), plDevList (421), plWireDevButtons (424), plWrkaPanel (431), plPush (466), plPoint (467), plNextCode (471), plRefreshCanvas (477), plWireEditor (486), plWireProps (499), plWireCanvas (518), plRotate (594), plDuplicate (595), plDelete (603), plUndoNow (604), plNewPlan (637)

`servicedesk/foto.js` 331 righe · 25 funzioni

Foto di cantiere e GPS.

Funzione	Riga	Cosa fa
fotoUpload(path, blob, retry = true)	34	Carica una foto ridotta nello Storage «foto».
fotoGridHtml(paths, opts = {})	67	griglia di miniature: riempita dopo con i link firmati
fotoPickerHtml(id, label = 'Aggiungi foto')	90	selettore di foto riutilizzabile (Off topic): restituisce le foto scelte e già compresse
fotoFlushQueue()	141	Invia le foto rimaste in coda senza rete.
ctElenco()	170	Elenco cantieri (servicedesk/cantieri.js) + quelli già usati nelle foto
ctTrovaVicino(compila)	205	Trova il cantiere più vicino con il GPS.
cantiereView()	226	Foto cantiere: progetto, GPS, nota, scatto.

Altre funzioni di supporto: fotoCompress (13), fotoUuid (30), fotoPath (31), fotoSign (48), fotoRemove (61), fotoHydrate (71), fotoLightbox (79), fotoPickerWire (96), fotoUploadMany (113), fotoQueueDb (120), fotoQueue (128), fotoSaveSitePhoto (160), ctDistanza (180), ctMetri (186), ctPiuVicino (187), ctGpsOn (196), ctPosizione (197), ctRender (249)

`servicedesk/fornitori.js` 177 righe · 9 funzioni

Rubrica fornitori e licenze della persona.

Funzione	Riga	Cosa fa
fornCanSee()	8	la rubrica la usano e la aggiornano tutti: resta scritto chi ha creato e chi ha modificato ogni fornitore
fornitoriView()	23	Rubrica fornitori.
fornForm(s)	68	Scheda fornitore (tutti modificano, resta scritto chi).
personSoftwarePanel(personId, personAssets)	124	Licenze, programmi e account di una persona (solo IT).

Altre funzioni di supporto: fornCanEdit (9), fornCanDelete (10), fornDefaultAmbito (16), fornSetupMsg (17), fornRender (34)

`servicedesk/push.js` 101 righe · 7 funzioni

Notifiche vere del browser (Web Push).

Funzione	Riga	Cosa fa
setupKeys()	21	Prima volta in assoluto: il Super admin crea le chiavi (restano nel database, mai nel codice)
window.archeaEnablePush()	59	dal pulsante (serve un tocco dell'utente per chiedere il permesso)
window.archeaPushSync()	77	a ogni avvio: se il permesso c'è già, rinnova l'iscrizione in silenzio
window.archeaPushBanner()	83	riquadro in Inizio finché le notifiche non sono attive

Altre funzioni di supporto: registration (13), publicKey (28), subscribe (35)

`assets/sd-shell.js` 176 righe · 11 funzioni

Collega Workspace e Service Desk: iframe unico, menu a sinistra, prima pagina del PM.

Funzione	Riga	Cosa fa
window.archeaSdHome()	160	Inizio: per il reparto IT la prima pagina è la Dashboard, per gli altri l'Inizio a cartelle

Altre funzioni di supporto: ensureFrame (12), window.archeaSdReady (27), flushPending (40), tick (47), goTab (93), run (97), navData (103), paint (119), syncActive (139), ensureNavMounted (141)

`assets/ws-cloud.js` 154 righe · 13 funzioni

Salva progetti, task, ore, note, squadre e organico del Workspace su Supabase (ws_records).

Funzione	Riga	Cosa fa
api(path, opt, retry)	16	Unico punto che parla con Supabase: aggiunge chiave e token, rinnova la sessione se scade (401) e trasforma gli errori in messaggi.
toast(msg)	33	Messaggio breve in basso.
loadStaff()	68	Organico: le persone vere dell'anagrafica HR del Service Desk (se l'account le può leggere)

Altre funzioni di supporto: cfg (12), token (13), emit (32), keyOf (34), loadAll (36), loadPeople (49), start (78), stop (91), persist (93), set (129)

`config/platform.js` 174 righe · 9 funzioni

Configurazione, accesso unico (ArcheaAuth) e tema (ArcheaTheme).

Altre funzioni di supporto: read (22), write (25), call (29), appRoleFor (51), loadProfile (57), mode (144), resolved (153), apply (154), set (164)

`sw.js` 85 righe · 0 funzioni

Service worker: memoria per l'uso offline, notifiche push, clic sulle notifiche.

`app/src` (React)

File	Cosa contiene
`App.tsx`	
`components/Icona.tsx`	
`components/Layout.tsx`	
`components/Ticket.tsx`	
`lib/auth.tsx`	
`lib/roles.test.ts`	
`lib/roles.ts`	Ruoli, permessi e menu: logica pura, senza React, così si può testare.
`lib/supabase.ts`	Chiave pubblica (publishable): può stare nel codice. La service_role NON va mai qui.
`lib/tema.ts`	
`lib/tickets.ts`	
`main.tsx`	Memoria dei dati: ogni elenco si scarica una volta e si riusa per 30 secondi,
`pages/Home.tsx`	
`pages/IT.tsx`	
`pages/Login.tsx`	
`pages/MieiTicket.tsx`	
`pages/NuovoTicket.tsx`	
`pages/TicketDettaglio.tsx`	

4. Il database

Ogni tabella ha la sicurezza per riga (RLS): il database decide chi legge e chi scrive, non lo schermo.

Tabelle create dalle migrazioni

Tabella	Colonne	Migrazione
access_group_members	group_id, user_id, created_at	20261106_utenti_grupp
access_groups	id, nome, descrizione, permessi, created_at	20261106_utenti_grupp
calendar_attendees	event_id, user_id, risposta, created_at	20261106_calendario.s
calendar_events	id, tipo, titolo, descrizione, luogo, resource_id, start_at, end_at, stato, organizer_id, organizer_nome, created_at, updated_at	20261106_calendario.s
calendar_resources	id, tipo, nome, sede, dettagli, attivo, created_at	20261106_calendario.s
chat_group_members	group_id, user_id, last_read_at, created_at	20261107_chat_gruppi.
chat_group_messages	id, group_id, sender_id, body, created_at	20261107_chat_gruppi.

chat_groups	id, nome, tipo, progetto_codice, created_by, created_at	20261107_chat_gruppi.
chat_messages	id, sender_id, recipient_id, body, created_at, read_at	20261106_chat.sql
floor_plans	id, nome, sede, prefisso, width, height, griglia, sfondo, elements, ordine, updated_at, updated_by_nome	20261106_planimetrie.
forum_poll_options	id, topic_id, testo, ord	20261106_sondaggi.sc
forum_poll_votes	option_id, topic_id, user_id, user_nome, created_at	20261106_sondaggi.sc
forum_polls	topic_id, domanda, multipla, chiude_at, created_at	20261106_sondaggi.sc
forum_replies	id, topic_id, corpo, author_id, author_nome, created_at	20261106_offtopic.sql
forum_topics	id, categoria, titolo, corpo, author_id, author_nome, pinned, locked, replies_count, last_activity_at, created_at	20261106_offtopic.sql
person_software	id, person_id, tipo, nome, versione, supplier_id, riferimento, asset_id, data_inizio, scadenza, stato, note, created_by, created_at, updated_at	20261108_licenze_forr
push_config	id, public_key, private_key, function_url, hook_secret, updated_at	20261108_push.sql
push_subscriptions	id, user_id, endpoint, p256dh, auth, device, created_at, last_seen	20261108_push.sql
scan_relay	id, user_id, session, code, created_at	20261107_scan_telefor
site_photos	id, cantiere, path, nota, taken_at, lat, lng, author_id, author_nome, created_at	20261106_foto.sql
suppliers	id, nome, categoria, ambito, referente, telefono, email, sito, indirizzo, partita_iva, note, progetti, created_by, created_at, updated_at	20261108_licenze_forr
ws_records	collection, id, data, created_by, updated_by, created_at, updated_at	20261107_progetti_tasl

Tabelle di base già presenti prima delle migrazioni: profiles, tickets, comments, ticket_history, notifications, assets, asset_history, asset_assignments, people, person_events, hr_movements, map_positions, floor_plans, material_bookings, appointments, reference_values...

Funzioni SQL (chiamabili con rpc o usate nelle regole)

Funzione	Parametri	Note
admin_users	p_action text, p jsonb default '{}'::jsonb	
calendar_notify_answer		
calendar_notify_cancel		
calendar_notify_invite		5) Notifiche (campanella): invito, risposta, annullamento
can_book_resource	rid bigint	
chat_contacts		3) Rubrica: solo id, nome e ruolo dei colleghi (non espone gli altri campi di profiles)
chat_group_add_creator		chi crea un gruppo ne fa subito parte
chat_group_notify		notifica a tutti gli altri del gruppo
chat_is_group_admin	gid bigint	chi crea un gruppo o una sottochat la gestisce (nome, persone, eliminazione)
chat_is_member	gid bigint	
chat_is_project	gid bigint	
chat_notify		4) Notifica al destinatario (campanella del Service Desk)
chat_sync_project		Chat di progetto: segue la squadra del progetto (ws_records, collection 'team')
forum_guard_reply		Risposte: l'autore può modificare il testo
forum_guard_topic		Argomenti: regole di modifica (sostituisce la versione precedente della funzione)
forum_on_reply		Contatore risposte, ultima attività e notifiche a chi partecipa
forum_on_reply_delete		
forum_poll_open	tid bigint	
forum_poll_single_choice		Scelta singola: un nuovo voto sostituisce il precedente
has_perm	p text	2) Funzioni di controllo
is_it_role		La vede e la modifica solo l'IT. NON si salvano le chiavi di licenza.
is_it_staff		
is_super_it		I gruppi che danno "gestione utenti" li modifica solo il super admin
my_permissions		I SUPER_IT hanno anche il permesso gestione_mappe (correzione di my_permissions)
profiles_guard_insert		Un profilo creato dal browser da chi non gestisce gli utenti nasce sempre USER
profiles_guard_role		Il ruolo IT o super admin lo assegna solo il super admin (anche fuori dal sito)
push_claim	p_endpoint text, p_p256dh text, p_auth text, p_device text d	salva l'iscrizione di questo dispositivo per chi è entrato (un dispositivo = l'ultimo che vi è entrato)
push_on_notification		A ogni notifica nuova: chiama la Edge Function (in coda, non rallenta e non blocca mai l'inserimento)
push_public_key		chiave pubblica: serve a ogni dispositivo per iscriversi
push_setup	p_public text, p_private text, p_url text, p_force boolean d	prima configurazione (solo Super admin). Non sovrascrive chiavi già presenti, salvo p_force.
push_status		stato per la pagina di controllo (solo Super admin)
supplier_can_edit	p_ambito text	
suppliers_stamp		
ws_assigned_to_me	assegnatario text	vero se il mio nome è tra gli assegnatari ("Mario Rossi, Anna Bianchi")

ws_guard		date, priorità e validazione.
ws_is_hr		HR e super admin: gli unici che vedono e modificano i dati dell'Organico (contratti, compensi)
ws_is_pm		3) Funzioni di controllo
ws_my_name		

Edge Functions

- `admin-users` : ARCHEA Platform · Edge Function "admin-users"
- `push` : ARCHEA Platform · Edge Function "push"
- `telegram-new-ticket` : avviso Telegram all'IT per ogni ticket (pubblicata su Supabase).

5. Comandi utili

Cosa	Comando
Nuova versione: sviluppo / test / pubblicazione	`cd app && npm run dev` · `npm test` · `npm run build`
Rigenerare lo script del database	`python3 tools/genera_setup.py`
Rigenerare tema scuro e contrasti	`python3 tools/genera_dark.py`
Rigenerare questo datasheet	`python3 tools/genera_datasheet.py`

6. Regole per chi scrive codice (AGENTS.md)

Questo file vale per **chiunque** modifichi il progetto: sviluppatori, Claude, ChatGPT, Copilot, Gemini o altre AI. Le regole sono **obbligatorie**. Se una richiesta le contraddice, si chiede prima a Giuseppe Milano (IT Archea). Autori del progetto: **Francesco Diamanti** (Workspace) e **Giuseppe Milano** (Service Desk, piattaforma, backend).

---

1. Com'è fatto il progetto

Cartella	Cosa contiene	Regola
`app/`	**Nuova versione** in React 18 + TypeScript + Vite, pubblicata in `v2/`	**Tutte le funzioni nuove si scrivono qui.**
`servicedesk/`	Service Desk attuale, JavaScript senza framework	Solo correzioni e piccoli ritocchi finché non è sostituito. Niente moduli nuovi grandi.
`assets/workspace.js`	Workspace di Francesco **compilato** (il sorgente aggiornato non esiste)	Non si estende. Solo patch minime, ognuna con un commento `// ARCHEA:` che spiega perché.
`assets/sd-shell.js` , `assets/ws-cloud.js` , `config/`	Collegamenti tra Workspace, Service Desk e Supabase	Modifiche piccole e commentate.
`supabase/migrations/`	Struttura del database, una migrazione per file	Vedi §4.
`supabase/functions/`	Edge Function (codice sul server)	Le chiavi segrete solo qui, mai nel sito.
`tools/`	Script che generano file (`genera_setup.py` , `genera_dark.py` , `genera_sizes.py`)	I file generati non si modificano a mano.
`docs/`	Documentazione	Da aggiornare quando cambia il funzionamento.

Il database è **Supabase** (PostgreSQL + Auth + Storage + Edge Functions). In futuro andrà su un server dello studio (WSL2 + Docker + Tailscale, vedi `docs/PROSSIMO_PASSO_INDIPENDENZA.md`): **non usare servizi esterni nuovi** che renderebbero più difficile il trasloco.

2. Framework e librerie (decisioni prese, non si cambiano senza accordo)

- **React 18 + TypeScript in modalità `strict` + Vite.** Niente Angular, Vue, Svelte, jQuery, Next.js.
- **React Router** con `HashRouter` (GitHub Pages non gestisce le rotte del server).
- **TanStack Query** per tutti i dati che arrivano dal database (memoria, ricarica, stato di caricamento).
- **`@supabase/supabase-js`** , con **un solo client** in `app/src/lib/supabase.ts` . Mai `fetch` diretto verso Supabase fuori da `src/lib/` .
- **CSS semplice con variabili** in `app/src/styles.css` (tema chiaro e scuro cambiano solo le variabili). Niente Tailwind, Bootstrap o librerie di componenti.
- **Vitest** per i test. Il browser di prova è Playwright, già installato: **non eseguire `playwright install`** .
- Una libreria nuova si aggiunge solo se risparmia molto lavoro, con versione esatta in `package.json` , e va scritta nel CHANGELOG.

3. Come si scrive il codice

- **Lingua**: testi a schermo, commenti, messaggi di errore e commit in **italiano**, semplice e chiaro. Nomi di variabili e funzioni in italiano o inglese, ma coerenti dentro lo stesso file.
- **TypeScript**: niente `any` . Ogni tabella del database ha il suo tipo in `src/lib/` .
- **Struttura di `app/src/`** :
- `lib/` : dati e logica senza React (accesso al database, regole, calcoli). Funzioni pure dove possibile, e quindi testate.
- `components/` : pezzi riusabili (un componente per file, nome con la maiuscola).
- `pages/` : una pagina per file.
- `App.tsx` : solo le rotte.
- **Ruoli e permessi**:
- il menu e la prima pagina di ogni ruolo stanno **solo** in `src/lib/roles.ts` ;
- nessun componente decide da solo chi vede cosa;
- lo schermo nasconde, ma **è il database che vieta** (§4).
- **Interfaccia** (decisioni di Giuseppe):
- ogni ruolo entra nella **sua** pagina: IT → Dashboard, HR → Statistiche HR, PM → Il lavoro di oggi, utente → le sue richieste;
- le funzioni del Service Desk stanno nella **barra in alto**, raggruppate (Ticket, Asset, Persone e spazi, Amministrazione);
- tutto il resto sta nel **menu a sinistra**, raggruppato per tipo, con tutte le voci visibili (niente «Altro» che nasconde);
- chat, calendario, Off topic e foto cantiere si aprono a schermo libero, senza barra sopra;
- **niente pagina a cartelle** all'inizio.
- **Accessibilità e telefono**:
- ogni campo ha la sua etichetta e i testi sono grandi;
- sul telefono i campi hanno almeno 16 px, altrimenti iPhone ingrandisce la pagina;
- contrasto almeno 4,5:1, anche nel tema scuro;
- ogni pagina ha la **favicon**.
- **Errori**: sempre un messaggio in italiano che dice cosa fare. Mai una schermata bianca.
- **Niente dati dimostrativi** negli account veri. La parte di Francesco non va nascosta, perché è la parte fondamentale.
- **Commenti**: spiegano il *perché*, non il *cosa*. In cima a ogni file una riga che dice a cosa serve.

4. Database: regole severe

1. **Ogni modifica è una migrazione nuova**: `supabase/migrations/AAAAMMGG_nome.sql`. Non si riscrivono le migrazioni già pubblicate, salvo per correggere un errore.
2. **Ogni migrazione si può rieseguire senza danni**: `create table if not exists`, `add column if not exists`, `create or replace function`, `drop policy if exists` prima di `create policy`.
3. **Ogni tabella nuova ha `enable row level security` e le sue regole (policy)**. Nessuna tabella senza RLS.
4. Le regole usano le funzioni già pronte:
5. `has_perm('…')`;
6. `is_it_role()`;
7. `ws_is_hr()`;
8. `ws_is_pm()`;
9. `chat_is_member(id)`.

Le funzioni nuove sono `security definer set search_path = public`, con `revoke … from public, anon` e `grant … to authenticated`. 5. Dentro una policy **non** si fa una sottoquery sulla stessa tabella con nomi di colonna ambigui: si usa una funzione `security definer`. Esempio: `chat_is_project()`. 6. Dopo una migrazione nuova: - si aggiunge il nome in `tools/genera_setup.py` (lista `ORDER`); - si lancia `python3 tools/genera_setup.py`; - si prova `supabase/SETUP_COMPLETO.sql` **due volte di seguito** su un PostgreSQL locale; - si prova la regola (policy) con almeno un utente che **può** e uno che **non può**. 7. Ruoli: `USER`, `HR`, `IT`, `SUPER_IT`. Permessi dei gruppi: - `gestione_utenti`; - `moderazione_offtopic`; - `gestione_calendario`; - `gestione_mappe`; - `gestione_progetti` (= PM).

Il vincolo `access_groups_permessi_validi` deve elencarli **tutti** in ogni migrazione che lo ricrea. 8. Ogni modifica importante lascia traccia: chi (`created_by` / `updated_by` + nome) e quando. Esempi: movimenti, fornitori, task.

5. Sicurezza e privacy (mai derogabili)

- **Mai** nel codice del sito:
- la chiave `service_role`;
- il token di Telegram;
- le chiavi VAPID private;
- password.

Nel sito va solo la chiave *publishable*, in `config/platform.js` e `app/src/lib/supabase.ts`. Le chiavi segrete stanno nelle Edge Functions o in tabelle senza accesso dal sito (come `push_config`). - **Mai** salvare chiavi di licenza software (`person_software.riferimento` è solo un numero di contratto o ordine). - Contratti e compensi dell'Organico: li vedono solo **HR** e **Super admin**. - Note interne IT sui ticket: solo IT. - Uno spostamento sulla planimetria richiede **due conferme**, poi la scrittura nel database e la **verifica rileggendo** il dato.

6. Come si aggiunge una parte nuova (passo per passo)

1. **Database**: migrazione nuova con tabella, RLS, policy e funzioni (§4). Rigenera `SETUP_COMPLETO.sql` e provalo.
2. **Dati**: in `app/src/lib/<modulo>.ts` scrivi il tipo della riga e le funzioni che leggono e scrivono (con `supabase.from(...)`).
3. **Regole**: se cambia chi vede cosa, aggiorna `app/src/lib/roles.ts` e i suoi test in `roles.test.ts`.
4. **Pagina**: `app/src/pages/<Nome>.tsx`, con `useQuery` e `useMutation` di TanStack Query; stati di caricamento ed errore sempre visibili.
5. **Rotta**: aggiungila in `app/src/App.tsx`.
6. **Menu**: aggiungi la voce in `gruppiMenu()` nel gruppo giusto (§3).
7. **Prove**:
8. `npm test` e `npm run typecheck`;
9. una prova nel browser con Supabase simulato, su computer e su telefono, con almeno due ruoli diversi;
10. **non dire mai** che una cosa è provata sul database vero se non lo è.
11. **Pubblicazione**: `npm run build`, che scrive in `v2/`, poi commit di `app/` **e** di `v2/`.
12. **Documentazione**:
13. una riga nel `CHANGELOG.md` con il numero di versione;
14. aggiorna la tabella in `app/README.md`;
15. se cambia l'uso, anche le guide in `docs/`.

Per il **sito attuale** (`servicedesk/`, `index.html`), solo correzioni: - quando cambi un file, aumenta il suo `?v=` in `index.html`; - aumenta `CACHE_NAME` in `sw.js`, così i telefoni scaricano la versione nuova.

7. Git

- Un commit per ogni modifica che ha senso da sola, con il messaggio in italiano e la versione: `3.1.0 Rubrica fornitori…`.
- Le AI aggiungono in fondo al messaggio la riga `Co-Authored-By:` con il proprio nome.
- Prima di dire «fatto»: commit **e** push.
- Mai forzare il push (`--force`) sul ramo principale.

8. Cose vietate

- Eseguire `playwright install`.
- Mettere segreti nel codice o nei commit.
- Disattivare l'RLS o scrivere policy `using (true)` per scrivere dati che non sono pubblici.
- Ripristinare la pagina a cartelle, il menu «Altro», gli account demo o i dati dimostrativi nel sito vero.
- Riscrivere da zero parti che funzionano senza che sia stato chiesto.

- Cambiare framework o librerie scelte (§2) di propria iniziativa.